

GISMA UNIVERSITY OF APPLIED SCIENCES

**Project Report:
Computer Science Laboratory
Portfolio**

Module: B201 Computer Science Lab

Student Name: Arjaf Anwar

Student ID / GH Code: GH1059387

Location: Berlin, Germany

Date: June 2026

Submitted in partial fulfillment of the assessment requirements for the Individual Project module.

Contents

1	Introduction and Digital Artifact Core Framework	2
2	Repository Architecture and Modular Directory Topologies	2
2.1	Structural Mapping of the Git File System	3
2.2	Separation of Concerns and Architectural Mapping Justification	3
3	Algorithmic Analysis of Core Programmatic Core Exercises	4
3.1	Functional Design of the Student Mark Sheet System: <code>grades.py</code>	4
3.2	Structural Mechanics of the Multi-Purpose Unit Converter: <code>Degree.py</code>	4
3.2.1	Menu Logic and Conditional Branching Paths	4
3.2.2	Data Processing and Compilation Behaviors	5
4	Justification of Interface Architecture and Visual Design Decisions	5
4.1	The Three-Stop Linear Balanced Background	5
4.2	Card Layout and Layered Contrast Properties	6
4.3	Interactive Control Mechanics and Typography	6
5	Exposition of Tools and Applied Engineering Technologies	6
5.1	The Structural Layer: HTML5 Core Constructs	6
5.2	The Layout Engine: Dynamic CSS3 Cascade Configurations	7
5.3	The Document Pipeline: Typesetting via LaTeX Engine Frameworks	7
5.4	The Staging Workspace: Cloud Infrastructure via GitHub Pages	7
6	Critical Self-Evaluation and Strategic Technical Roadmap	8
6.1	Core Portfolio Strengths	8
6.2	Portfolio Limitations and Structural Weaknesses	8
6.3	Strategic Future Enhancement Roadmap	8
6.3.1	Interactive In-Browser Execution via WebAssembly	8
6.3.2	Dynamic Content Loading via REST API and Data Structures	9
7	Conclusion	9

1 Introduction and Digital Artifact Core Framework

In the contemporary landscape of global software engineering, a developer's digital infrastructure serves as an essential medium for verifying operational capability, technical literacy, and design philosophy. Traditional resumes offer limited visibility into a developer's systematic approaches to version control, layout stability, or semantic code compilation. To address this limitation, this comprehensive technical report documents the complete structural design, layout engineering, and environment configuration involved in building an automated, cloud-hosted Computer Science Portfolio webpage. Developed to fulfill the rigorous assessment criteria of the B201 Computer Science Lab module at Gisma University of Applied Sciences, this project establishes a functional, accessible showcase for prospective technical recruiters and hiring managers within the software industry.

The digital ecosystem engineered for this module operates as a unified platform containing production-ready scripts, curated coursework documentation, and structured layout styling sheets. This portfolio is deployed utilizing a distributed edge computing model via GitHub Pages, which automates static resource rendering directly from a version-controlled repository pipeline. The corresponding live artifacts can be accessed dynamically via the following production addresses:

- **Live Active Portfolio Deployment Web Link:**
<https://arjaf-anwar.github.io/CS--Portfolio-/>
- **Production Git Repository Version Workspace Link:**
<https://github.com/Arjaf-Anwar/CS--Portfolio->

By establishing these public endpoints, the project demonstrates foundational competency in configuring web domains, managing file trees, and deploying applications within an isolated staging environment. The subsequent sections of this document provide an in-depth analysis of the structural mapping, design choices, programmatic logic, and hosting tools applied during the engineering lifecycle of this project.

2 Repository Architecture and Modular Directory Topologies

A primary indicator of professional software maturity is the organization of a project's source architecture. Monolithic directory structures—where scripts, image configurations, style specifications, and raw markup are pushed indiscriminately into a single root folder—result in technical debt, dependency friction, and deployment errors. To preserve scalability and maintain clear separation of concerns, a strict modular directory topology was designed and enforced within the development repository workspace.

2.1 Structural Mapping of the Git File System

The physical organization of files in the project workspace is configured according to a precise hierarchical tree model. The root directory contains the entry point markup container file named `index.html`, which establishes the structural skeleton of the portfolio. Accompanying this is the presentation ruleset stylesheet file named `style.css`. To display the owner's professional identity cleanly, the binary image file named `Arjaf anwar passport size pic.jpg` is positioned within the root workspace, granting the hypertext engine immediate access during rendering cycles.

To separate project sub-components from the primary delivery layers, two dedicated directories were created:

- **The Document Sub-Directory Area (`cv/`):** This directory acts as an isolated document workspace housing the personal curriculum vitae files. It contains the raw native document source file named `cv cs.tex` alongside the compiled vector output binary file named `cv cs.pdf`.
- **The Computational Execution Sub-Directory Workspace (`projects/`):** This directory isolates programmatic logic from the layout components. It serves as a secure script repository containing two terminal-based scripts: the procedural academic manager named `grades.py` and the interactive unit calculator named `Degree.py`.

2.2 Separation of Concerns and Architectural Mapping Justification

This layout strategy directly implements the principles of Separation of Concerns. The structural layer (HTML5) defines semantic data points; the presentation layer (CSS3) dictates visual spacing and color systems; the programmatic layer (Python 3) implements command-line algorithmic utilities; and the typesetting layer (LaTeX) generates stable vector documentation.

Isolating the programmatic terminal scripts inside the `projects/` folder prevents the static web server from trying to execute raw backend scripts as part of standard frontend rendering cycles. Similarly, sequestering LaTeX documents within the `cv/` folder ensures that during updates, compiling document changes will not interfere with the core assets required by the live website. This organization guarantees that external auditors, such as university markers or corporate engineers, can quickly inspect specific project layers without sifting through unrelated configuration files.

3 Algorithmic Analysis of Core Programmatic Core Exercises

To demonstrate foundational software engineering capabilities within the portfolio, two distinct stand-alone backend scripts were created. These applications focus on core programming principles, including parameter handling, input typecasting, mathematical validation, and structured branching.

3.1 Functional Design of the Student Mark Sheet System: `grades.py`

The first terminal application, `grades.py`, handles data manipulation and score accumulation. In software design, managing user data streams requires strict verification bounds to prevent calculations from failing due to improper data types. The script establishes a procedural execution loop that requests academic numeric markers from the terminal command-line prompt.

Once these inputs are received, the system runs explicit type-conversion operations to transform standard text entries into memory-allocatable float parameters. The software logic then evaluates the numeric floats against specific academic performance criteria. It computes weighted averages and determines the appropriate grade bracket via multi-tiered conditional statements. This program demonstrates solid control over variables, data type safety, loop control parameters, and command-line input-output handling.

3.2 Structural Mechanics of the Multi-Purpose Unit Converter: `Degree.py`

The second application, `Degree.py`, focuses on menu branching logic and precise mathematical calculations. The script initializes an interactive choice menu that prompts users to select one of three specific operational modes: temperature conversion, distance evaluation, or system exit.

3.2.1 *Menu Logic and Conditional Branching Paths*

The software processes user selection input using strict string matching rules. If the user selects the first option, the engine targets a temperature conversion sequence. The script prompts the user for a value in Celsius, captures the string, and typecasts it into a floating-point decimal. It then processes the value through standard conversion math—multiplying the parameter by nine-fifths and adding thirty-two—before printing the converted value in Fahrenheit.

When the user selects the second option, the application triggers a distance conversion routine. It captures a value in kilometers, typecasts it into a floating-point number, and multiplies it by a conversion constant of zero point six two one three seven one. The system then processes the resulting value through a rounding function, limiting it to two decimal places to ensure clean data display.

The third option handles a clean exit sequence, printing a completion message and ending script execution. If the user submits an input outside the valid range, a final conditional block alerts them to the invalid choice and guides them to restart the application, which keeps the system stable and prevents unexpected crashes.

3.2.2 Data Processing and Compilation Behaviors

During execution, these scripts utilize the computer’s terminal shell environment. When a user runs a script directly from a local desktop browser link, the operating system opens a brief command window instance that automatically shuts down as soon as the final logic statement finishes.

To audit the underlying code properly, developers open these files using an Integrated Development and Learning Environment, such as Python’s native IDLE system. Running the modules from within IDLE keeps the session active, allowing evaluators to thoroughly analyze variable states, conditional branches, and data-type changes. This format satisfies the functional programming requirements outlined in the B201 curriculum.

4 Justification of Interface Architecture and Visual Design Decisions

Frontend design should balance visual appeal with absolute clarity. Poor font contrast, messy layouts, and low visibility can alienate visitors, reducing the effectiveness of a developer’s portfolio. To avoid these issues, the user interface was engineered using clean layout constraints, dark-mode design best practices, and high-contrast styling properties.

4.1 The Three-Stop Linear Balanced Background

The visual system centers around a custom three-stop linear background gradient that flows horizontally across the display viewport. The style sheet sets a gradient that transitions from a dark night-sky charcoal hue at the left edge, blends smoothly into a rich sapphire navy-blue shade in the center, and returns to the deep charcoal tone at the right edge.

This three-stop color system provides distinct advantages over single-color or dual-color backgrounds:

- **Visual Spotlighting:** The lighter center blue acts as a natural visual spotlight, drawing the viewer’s focus directly toward the central content cards.
- **High-Contrast Legibility:** The dark blue background creates an ideal canvas for light white and cyan text typography, reducing eye strain during long code review sessions.

- **Modern Look:** The smooth, dark color transitions provide a clean, modern aesthetic similar to popular premium software engineering IDEs and developer portals.

4.2 Card Layout and Layered Contrast Properties

The portfolio structure organizes content inside distinct container panels styled under the content block class. These container panels use an ultra-dark background color to create clear separation from the shifting background gradient. This design choice produces a crisp, layered visual hierarchy, framing text entries within clear borders.

The title areas within each content section use an electric cyan accent color. This color choice instantly highlights section breaks, making it easy for users to scan the portfolio. To ensure the student identification code remains completely readable against the dark background, it uses an electric cyan color combined with a text-shadow neon glow. This glow adds a subtle layer of illumination that separates the metadata text from the dark canvas below, keeping critical tracking details highly legible.

4.3 Interactive Control Mechanics and Typography

The resume interaction area utilizes two styled link objects designed as modern pill buttons. The primary download button features a bright cyan fill color paired with dark text typography, creating an immediate call to action for downloading the curriculum vitae document.

Positioned right beside it, the secondary LaTeX button utilizes a transparent background framed by a subtle border outline. When a user hovers over these buttons, interactive transition properties change the background colors and apply soft dropshadow highlights over a fraction of a second. This subtle animation confirms the buttons are active and responsive.

The typography uses clean sans-serif system fonts to ensure fast loading times and sharp presentation on both desktop monitors and high-density mobile screens.

5 Exposition of Tools and Applied Engineering Technologies

Building this portfolio required combining several core technologies, frameworks, and deployment engines. This multi-layered tool approach ensures the portfolio remains responsive, cross-browser compatible, and easy to maintain.

5.1 The Structural Layer: HTML5 Core Constructs

The portfolio framework relies entirely on semantic HTML5 markup. Instead of grouping all data points inside generic layout divs, the page utilizes structural tags such as header, section, and main. This design choice provides search engine crawlers and screen readers

with a clear understanding of the site hierarchy, optimizing accessibility and ensuring the site passes modern validation standards.

5.2 The Layout Engine: Dynamic CSS3 Cascade Configurations

The visual design uses advanced CSS3 properties to align components cleanly across the layout. The document positioning rules rely on the Flexible Box Layout engine, which automatically manages centering, spacing, and grid padding configurations.

To maintain presentation quality across diverse device shapes, the stylesheet includes responsive media query breaks. When a visitor views the site on a small smartphone display, the media query triggers, switching the header layout from a wide horizontal view to a stacked vertical alignment. This structural shift repositions the profile picture cleanly above the introductory text, preventing component clipping and layout brokenness on mobile browsers.

5.3 The Document Pipeline: Typesetting via LaTeX Engine Frameworks

The curriculum vitae documentation is built using the LaTeX typesetting language rather than standard word processing software. Standard word processors often introduce layout errors, uneven margins, and font variations when opened across different software versions.

In contrast, the LaTeX engine relies on declarative source code to control text layout, margins, and document formatting. This code compiles directly into a high-fidelity vector PDF, ensuring that line breaks, spacing, and structural components display perfectly on any screen or print medium.

5.4 The Staging Workspace: Cloud Infrastructure via GitHub Pages

The portfolio staging pipeline uses Git version control integrated directly with the GitHub Pages deployment platform. Git monitors change histories, tracks modifications across text files, and creates distinct commit restore points.

Once changes are verified and pushed to the online repository, automated background webhooks trigger a container build process. This build system reads the updated code repository, processes the cascade styling rules, and deploys the new static build to the cloud server network within seconds. This cloud pipeline eliminates the need for expensive web hosting packages or manual file transfer protocols, demonstrating modern web development capabilities.

6 Critical Self-Evaluation and Strategic Technical Roadmap

Analyzing a completed software project through a critical self-evaluation is an effective way to identify structural strengths, fix underlying weaknesses, and map out long-term functional upgrades.

6.1 Core Portfolio Strengths

The portfolio layout delivers several key development advantages:

- **Clean Architecture Separation:** Keeping markup, style rules, documentation files, and computational scripts completely separate makes the code easy to read and maintain over time.
- **Polished Visual Identity:** The custom three-stop gradient background provides a unique, professional visual style while keeping text content highly legible.
- **Full Mobile Responsiveness:** Flexbox alignment properties scale layouts cleanly across smartphones, tablets, and wide desktop screens without breaking components.

6.2 Portfolio Limitations and Structural Weaknesses

Despite these strengths, the current architecture has a few operational limitations:

- **Static Script Boundaries:** Visitors cannot execute or interact with files like `Degree.py` directly within the webpage. They must download the code and run it inside a local terminal environment.
- **Manual Content Updates:** Adding new projects or modifying profile details requires editing the raw HTML markup directly, which increases the risk of introducing syntax errors during updates.

6.3 Strategic Future Enhancement Roadmap

To address these limitations, the next development phase will focus on adding two primary feature enhancements to the platform:

6.3.1 *Interactive In-Browser Execution via WebAssembly*

The first upgrade will integrate an in-browser compilation framework, such as PyScript or Pyodide, which uses WebAssembly architectures. This enhancement will allow the webpage to run Python runtimes directly inside the visitor's browser engine.

Instead of downloading files like `Degree.py` to run locally, visitors will be able to input values and see real-time calculation outputs directly within an interactive terminal console embedded on the portfolio page.

6.3.2 Dynamic Content Loading via REST API and Data Structures

The second upgrade will replace the static HTML content lists with an asynchronous JavaScript fetching structure that pulls data from an external JSON file.

By shifting project descriptions and metadata to a separate data file, the portfolio can dynamically load new assignments onto the page using background API requests. This change means developers will no longer need to alter the core HTML file to update content, making the entire platform much more scalable and easier to manage.

7 Conclusion

The computer science portfolio engineered for this module successfully fulfills all technical, design, and structural requirements outlined in the B201 module assessment brief. By separating concerns across dedicated files, using a responsive three-stop linear background gradient, and organizing assets within clean directory trees, the system functions as a highly professional development showcase. This project successfully demonstrates core competencies in cloud deployment, frontend layout management, and structured document compilation, providing a dependable baseline for launching future software engineering applications.